



第一章 数据库系统概述

数据管理技术发展的三个阶段

阶段	核心特点	主要问题/优势
人工管理阶段	没有操作系统和数据管理软件；数据不长期保存在计算机中；数据组织由程序员负责；数据面向程序	数据和程序强绑定，几乎没有共享性
文件系统阶段	有操作系统和文件系统；以文件为单位长期保存数据；数据存取以记录为单位；数据与程序有一定独立性	独立性仍然差；共享性差、冗余大；容易不一致；查询和完整性维护困难
数据库系统阶段	由 DBMS 统一管理数据；数据面向整个系统；结构化、共享性好、冗余少、独立性高	需要 DBMS 提供安全性、完整性、并发控制、恢复等统一控制

例子：

- 文件系统阶段：教务处有一个学生文件，图书馆也有一个学生文件。学生改名时两个文件都要改，漏改就会数据不一致。
- 数据库系统阶段：学生信息统一放在数据库中，教务系统、图书馆系统通过 DBMS 访问同一份数据，减少重复和不一致。

数据库系统的组成

数据库系统通常包括：

- DB：数据库。统一管理、长期存储、可共享的数据集合。
- DBMS：数据库管理系统。数据库系统的核心软件，介于用户和操作系统之间。
- 用户：数据库管理员 DBA、终端用户、应用程序员。

容易混淆：

- 数据库 DB 是数据本身。
- 数据库管理系统 DBMS 是管理数据库的软件。
- 数据库系统 DBS 是 DB、DBMS、用户、软硬件环境的整体。

数据、信息、语法、语义

- 数据是信息的符号表示或载体。
- 信息是数据的内涵，是对数据语义的解释。
- 语法：数据格式，例如数值型、字符型、日期型。
- 语义：数据代表什么含义。

例子：<02055001, 张明, 男, 1984.5, 计算机系> 这串数据本身只是符号；如果知道每一列分别是学号、姓名、性别、出生年月、院系，才有语义。

数据模型三要素

数据模型是对现实世界数据特征的抽象，三要素是重点：

- 1. 数据结构：描述数据对象以及对象之间的联系。例如关系模型用二维表表示实体和联系。
- 2. 数据操作：描述可以对数据做什么操作，主要包括检索和更新，更新又包括插入、删除、修改。
- 3. 数据完整性约束：描述数据必须满足的规则。例如主键不能为空且不能重复，外键必须引用已有记录或为空。

例子：学生选课系统中，学生表、课程表、选课表属于数据结构；查询某学生成绩、插入一条选课记录属于数据操作；成绩必须在 0 到 100 之间属于完整性约束。

数据抽象层次与数据模型级别

现实世界到数据库中的数据通常经过三层抽象：

- 现实世界：真实存在的对象和业务，例如学生、课程、选课。
- 信息世界：概念模型，例如 E-R 模型。
- 机器世界：DBMS 支持的数据模型，例如关系模型、层次模型、网状模型。

数据模型按使用目的可分为：

- 概念数据模型：面向用户和现实世界，与具体 DBMS 无关。例如 E-R 模型。
- 逻辑数据模型：用户从数据库系统看到的数据模型，与 DBMS 相关。例如关系模型。
- 物理数据模型：反映数据如何存储。

模式、实例与三级模式结构

模式 Schema：

- 数据库中全体数据的逻辑结构和特征的描述。
- 只描述“型”，不涉及具体值。

实例 Instance：

- 模式在某一时刻的具体值。

例子：

- 模式：学生(学号, 姓名, 年龄)
- 实例：(2024001, 张三, 20)\$、\$(2024002, 李四, 19) 这些具体记录。

三级模式结构：

层次	又称	含义	数量
外模式	子模式、用户模式、视图层	某类用户能看到和使用的局部数据视图	一个数据库可有多 个
模式	逻辑模式	全体数据的逻辑结构和特征，是所有用户的公共数据视图	一个数据库只有一 个
内模式	存储模式、物理层	数据在数据库内部的物理组织和存储方式	一个数据库只有一 个

两级映像：

- 外模式/模式映像：保证逻辑独立性。模式改变时，调整映像使外模式不变，应用程序尽量不用改。
- 模式/内模式映像：保证物理独立性。存储结构改变时，调整映像使模式不变，应用程序不用改。

例子：

- 物理独立性：数据库从顺序存储改成 B+ 树索引存储，用户写的 SQL 不需要改变。
- 逻辑独立性：学生表增加一列 **电话**，某个只查询 **学号、姓名** 的外模式保持不变，对应程序不需要改。

DBMS 的主要功能

DBMS 是管理数据库的软件系统，主要功能包括：

- 提供用户接口：程序员用户、终端用户、DBA。
 - 数据库运行管理：安全性、完整性、并发控制、故障恢复、查询优化。
 - 数据目录管理：管理元数据，即“关于数据的数据”。
-

第二章 关系模型

关系模型的基本概念

概念	含义	例子
域 Domain	一组相同类型值的集合	性别域 {男, 女} ，年龄域 {18, 19}
笛卡尔积	多个域所有可能组合的集合	姓名域 × 性别域 × 年龄域
元组 Tuple	笛卡尔积中的一个元素，也就是表中的一行	(张三, 男, 20)
分量 Component	元组中的某个值	张三 、 男 、 20
关系 Relation	笛卡尔积的任意子集，直观上是一张二维表	学生表
属性 Attribute	关系中列的名字	学号、姓名、年龄
关系模式	对关系“型”的描述	Student(Sno, Sname, Sage)
关系	关系模式某一时刻的值	Student 表中当前所有学生记录

关系模式是静态的，关系是动态的。表结构通常相对稳定，表中的记录会随插入、删除、修改而变化。

键：超键、候选键、主键、外键

- 超键 SK：能唯一标识一个元组的属性或属性集。
- 候选键 CK：最小的超键，去掉任一属性后就不能唯一标识元组。
- 主键 PK：从候选键中选定的一个键。
- 外键 FK：关系 R1 的某个属性集引用另一个关系 R2 的候选键，则该属性集是 R1 的外键。

例子：

Student(Sno, Sname, Sage)
Course(Cno, Cname, Credit)
SC(Sno, Cno, Grade)

- Student 中 Sno 可以唯一标识学生，是候选键，也常作为主键。
- (Sno, Sname) 也能唯一标识学生，但不是最小的，所以是超键，不是候选键。
- SC.Sno 引用 Student.Sno，所以 SC.Sno 是外键。
- SC.Cno 引用 Course.Cno，所以 SC.Cno 是外键。
- SC\$的主键通常是\$(Sno, Cno)，表示一个学生选一门课只有一条记录。

关系的特征

关系模型中的关系有这些特征：

- 元组不重复且无序。
- 属性不重名且无序。
- 同一属性下的值来自同一域，即同一类型。
- 每个分量必须是原子的，不允许“表中有表”。

- 属性值可以为空值 **NULL**，表示未知或不适用。

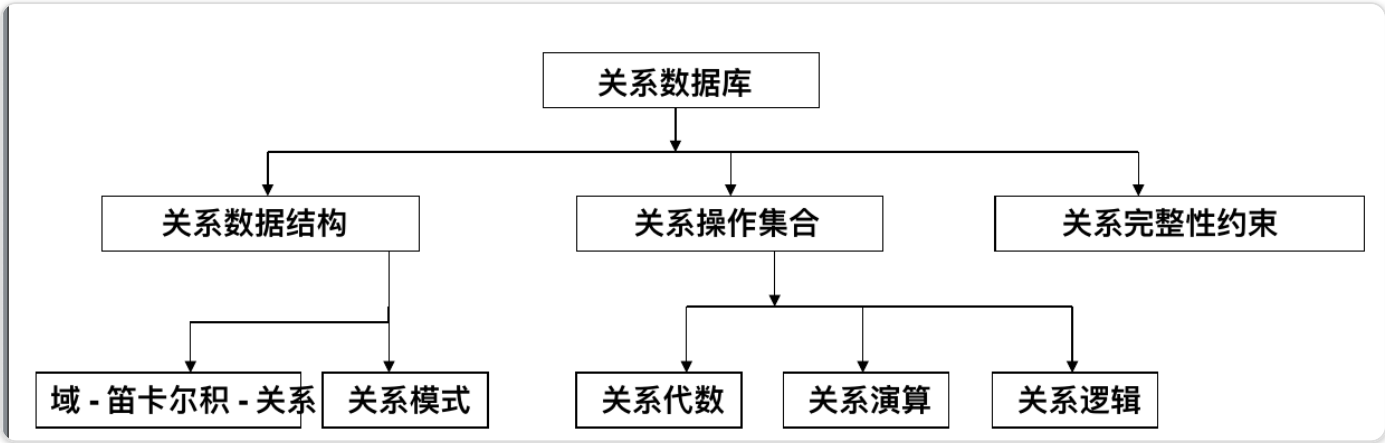
考试易错点：投影后如果出现重复元组，要合并去重，因为关系不允许重复元组。

关系完整性约束

类型	要求	例子
域完整性	属性取值必须在其值域内，是否允许 NULL 由语义决定	成绩应在 0-100 或 NULL
实体完整性	主键值不可重复，且不能为 NULL	学号不能重复、不能为空
参照完整性	外键值要么等于被引用表中某个候选键值，要么为 NULL	SC.Sno 必须存在于 Student.Sno
用户定义完整性	具体应用的业务规则	选课成绩只能是 0-100；性别只能是男/女

关系模型三要素

关系模型由三部分组成：



1. 关系数据结构：二维表、域、笛卡尔积、关系、关系模式。
2. 关系操作集合：关系代数、关系演算，以及查询、插入、删除、修改等操作。
3. 关系完整性约束：域完整性、实体完整性、参照完整性、用户定义完整性。

这和第一章“数据模型三要素”是一一对应的：数据结构、数据操作、完整性约束。

关系代数总览

关系代数的特点：运算对象是关系，运算结果仍是关系。

关系代数运算大致分两类：

类型	运算符	含义
传统集合运算	$\cup$	并
传统集合运算	$-$	差
传统集合运算	$\cap$	交
传统集合运算	$\times$	广义笛卡尔积
专门关系运算	σ	选择，筛选行
专门关系运算	π	投影，筛选列并去重
专门关系运算	$\bowtie$	连接，把相关元组拼接起来
专门关系运算	∇	除，表达“全部”“所有”

并、差、交要求两个关系“并兼容”：属性个数相同，且对应属性来自相同域。

$$R(A_1, A_2, \dots, A_n), \quad S(B_1, B_2, \dots, B_n)$$

如果 A_i 和 B_i 对应的域相同，则 R 和 S 可以做 $\cup$ 、 $-$ 、 $\cap$ 。

笛卡尔积 $\times$ 不要求并兼容。若 R 有 m 个属性、 $|R|$ 个元组， S 有 n 个属性、 $|S|$ 个元组，则：

$$R \times S \text{ 有 } m + n \text{ 个属性, 且 } |R \times S| = |R| \cdot |S|$$

注意：笛卡尔积不会自动合并同名属性。如果两个关系都有属性 A ，结果里要用 $R.A$ 、 $S.A$ 或重命名来区分。

选择、投影、连接、除

选择 σ

从关系中选出满足条件的行。

一般形式：

$$\sigma_F(R) = \{t \mid t \in R \wedge F(t)\}$$

例子：

$$\sigma_{\text{Dept}=\text{CS}}(\text{Student})$$

含义：选择计算机系学生。

投影 π

从关系中选出若干列，并去掉重复元组。

一般形式：

$$\pi_X(R) = \{t[X] \mid t \in R\}$$

例子：

$$\pi_{\text{Sno}, \text{Sname}}(\text{Student})$$

含义：只保留学生的学号和姓名。

投影的结果是集合，所以重复行会被消去。例如两个学生重名， $\pi_{\text{Sname}}(\text{Student})$ 里同一个姓名只出现一次。

连接 $\bowtie$

连接可以看作“笛卡尔积 + 选择”。

θ 连接

θ 连接是在笛卡尔积后按某个比较条件筛选， θ 可以是 $=$ 、 $>$ 、 $<$ 、 $\geq$ 、 $\leq$ 、 $\neq$ 。

$$R \bowtie_{A\theta B} S = \sigma_{R.A\theta S.B}(R \times S)$$

例子：

$$\text{Student} \bowtie_{\text{Student.Sno}=\text{SC.Sno}} \text{SC}$$

含义：把学生表和选课表按学号匹配起来。

等值连接

等值连接是 θ 连接的特殊情况，连接条件只使用等号。

$$R \bowtie_{R.A=S.B} S = \sigma_{R.A=S.B}(R \times S)$$

注意：等值连接通常保留两边的连接属性。例如

$R.A$ 和 $S.B$ 虽然值相等，但结果里仍可以同时保留 $R.A$ 和 $S.B$ 。

自然连接

自然连接是“按所有同名属性相等连接，并且同名属性只保留一份”。

设：

$$C = \text{Attr}(R) \cap \text{Attr}(S)$$

则自然连接可写成：

$$R \bowtie S = \pi_{\text{Attr}(R) \cup (\text{Attr}(S) - C)} \left(\sigma_{\bigwedge_{A \in C} R.A=S.A} (R \times S) \right)$$

如果只有一个同名属性 A ，就退化为 PPT 里的形式：

$$R \bowtie S = \pi_{\text{Attr}(R) \cup (\text{Attr}(S) - \{A\})} (\sigma_{R.A=S.A} (R \times S))$$

这个公式分三步看：

1. 先做 $R \times S$ ，得到所有可能配对。
2. 再做 $\sigma_{R.A=S.A}$ ，只保留同名属性值相等的配对。
3. 最后做 π ，把重复出现的同名属性删掉，只保留一份。

例子：

$$\text{Student}(\text{Sno}, \text{Sname}) \bowtie \text{SC}(\text{Sno}, \text{Cno})$$

自然连接会按 Sno 相等连接，结果模式是：

$$(Sno, Sname, Cno)$$

如果两个关系属性名和数量完全相同，且每个同名属性语义也相同，则自然连接等价于交：

$$R \bowtie S = R \cap S$$

但如果只是“碰巧同名”，自然连接可能会错连。考试写关系代数时，条件不明确的情况下，等值连接比自然连接更不容易产生歧义。

外连接

外连接不是最基本的传统关系代数运算，但和 SQL 很相关。

类型	含义
左外连接	保留左关系 R 中所有元组；右边匹配不到的属性补 \$NULL\$
右外连接	保留右关系 S 中所有元组；左边匹配不到的属性补 \$NULL\$
全外连接	左右两边匹配不到的元组都保留，缺失部分补 NULL

内连接只保留能匹配上的元组，外连接会额外保留某一侧或两侧未匹配的元组。

半连接

半连接只保留左关系中能与右关系匹配的元组，并且结果只含左关系的属性。

$$R \ltimes S = \pi_{Attr(R)}(R \bowtie S)$$

例子：

$$Student \ltimes SC$$

含义：保留至少有选课记录的学生，但结果里只保留 Student 的属性。

自连接

自连接是一个关系和它自己的副本连接。做题时必须重命名，否则左右两份属性分不清。

例子：查询至少选修两门课程的学生。

$$\pi_{A.Sno}(\sigma_{A.Sno=B.Sno \wedge A.Cno \neq B.Cno}(\rho_A(SC) \times \rho_B(SC)))$$

$\rho_A(SC)$ 和 $\rho_B(SC)$ 表示把 SC 重命名成两个副本，条件 $A.Sno = B.Sno$ 保证是同一个学生，条件 $A.Cno \neq B.Cno$ 保证是两门不同课程。

除 ∇

除法常用于“全部”“所有”类查询。

给定：

$$R(X, Y), \quad S(Y)$$

其中 X 和 Y 都可以是一组属性。 $R \nabla S$ 的结果只保留 X 属性：

$$R \nabla \cdot S = \{x \mid x \in \pi_X(R) \wedge \forall y \in S, (x, y) \in R\}$$

这句话的意思是：找出那些 x ，它和 S 中每一个 y 的组合都出现在 R 里。

PPT 里的 $t[X]s$ 表示“把元组 t 在 X 上的分量，和 S 中元组 s 拼成一个新元组”。例如：

$$t[X] = (S1), \quad s = (C1)$$

则：

$$t[X]s = (S1, C1)$$

所以 PPT 公式可以理解为：

$$T = R \nabla \cdot S = \{t[X] \mid t \in R \wedge \forall s \in S, t[X]s \in R\}$$

也就是说，某个 $t[X]$ 能进结果，当且仅当它和 S 里的每个 s 拼起来之后，都能在 R 里找到。

除法也可以用基本关系代数表示：

$$R \nabla \cdot S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

这个公式是最适合手算的版本：

1. $\pi_X(R)$ ：列出所有候选 x 。
2. $\pi_X(R) \times S$ ：列出每个候选 x 理论上应该具备的全部 (x, y) 组合。
3. $(\pi_X(R) \times S) - R$ ：找出缺失的组合。
4. $\pi_X((\pi_X(R) \times S) - R)$ ：找出有缺失组合的“坏候选” x 。
5. $\pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$ ：所有候选减去坏候选，剩下的就是满足“全部”的 x 。

也可以用“象集”理解。对每个 x ，令：

$$Y_x = \{y \mid (x, y) \in R\}$$

那么：

$$x \in R \nabla \cdot S \iff S \subseteq Y_x$$

例子：SC 表示学生选课。

Sno	Cno
S1	C1
S1	C2
S2	C1
S3	C1
S3	C2

S 表示要求选过的课程：

Cno
C1
C2

则：

$$SC(Sno, Cno) \nabla \cdot S(Cno) = \{S1, S3\}$$

因为：

$$Y_{S1} = \{C1, C2\}, \quad Y_{S2} = \{C1\}, \quad Y_{S3} = \{C1, C2\}$$

$S1$ 和 $S3$ 的课程集合都包含 $\{C1, C2\}$ ， $S2$ 不包含。

典型考试表达式：

$$\pi_{Sno, Cno}(SC) \nabla \cdot \pi_{Cno}(Course)$$

含义：选修了全部课程的学生学号。

这里：

$$X = \{Sno\}, \quad Y = \{Cno\}$$

被除数 $\pi_{Sno, Cno}(SC)$ 表示“谁选了哪门课”，除数 $\pi_{Cno}(Course)$ 表示“全部课程”。结果就是那些对每门课都存在选课记录的学生。

自然语言转关系代数的做题套路

统一使用这个模式：

Student(Sno, Sname, Sex, Age)
 Course(Cno, Cname, Teacher)
 SC(Sno, Cno, Grade)

步骤：

1. 先判断要输出什么列，用 π 。
2. 再判断筛选条件，用 σ 。
3. 如果条件或输出跨多个表，先连接相关关系。
4. 出现“不”“未”“没有”，优先考虑差 — 或 **NOT EXISTS** 思路。
5. 出现“至少两门”“同时选 C2 和 C4”，常用同一关系自乘或自连接。
6. 出现“所有”“全部”，优先考虑除 $\nabla \cdot$ 。

常见题型：

查询王老师开设课程的编号和课程名

$$\pi_{Cno, Cname} (\sigma_{Teacher=王}(Course))$$

先在 Course 中选教师为王的课程，再投影课程编号和课程名。

查询至少选修了一门王老师课程的学生学号

写法 1:

$$\pi_{\text{Sno}}(\sigma_{\text{Teacher}=\text{王}}(\text{SC} \bowtie \text{Course}))$$

写法 2:

$$\pi_{\text{Sno}}(\text{SC} \bowtie \pi_{\text{Cno}}(\sigma_{\text{Teacher}=\text{王}}(\text{Course})))$$

第二种写法更清楚：先找王老师课程编号，再和选课表连接。

查询李同学未选课程的编号

$$\pi_{\text{Cno}}(\text{Course}) - \pi_{\text{Cno}}(\sigma_{\text{Sname}=\text{李}}(\text{Student} \bowtie \text{SC}))$$

含义：所有课程编号减去李同学已经选的课程编号。

查询至少选修两门课程的学生学号

$$\pi_1(\sigma_{1=4 \wedge 2 \neq 5}(\text{SC} \times \text{SC}))$$

这里把 $\text{SC}(\text{Sno}, \text{Cno}, \text{Grade})$ 自乘：

- 左边 SC 的第 1 列是学生号，第 2 列是课程号。
- 右边 SC 的第 4 列是学生号，第 5 列是课程号。
- $1 = 4$ 表示同一个学生。
- $2 \neq 5$ 表示两门不同课程。

更易读的写法：

$$\pi_{A.\text{Sno}}(\sigma_{A.\text{Sno}=B.\text{Sno} \wedge A.\text{Cno} \neq B.\text{Cno}}(\rho_A(\text{SC}) \times \rho_B(\text{SC})))$$

其中 $\rho_A(\text{SC})$ 和 $\rho_B(\text{SC})$ 表示把同一个 SC 关系分别重命名为两个副本，方便区分左右两边的属性。

查询全部学生均选修了的课程编号及名称

$$\pi_{\text{Cno}, \text{Cname}}(\text{Course} \bowtie (\pi_{\text{Sno}, \text{Cno}}(\text{SC}) \nabla \cdot \pi_{\text{Sno}}(\text{Student})))$$

解释：

- $\pi_{\text{Sno}, \text{Cno}}(\text{SC})$ 表示“谁选了哪门课”。
- $\pi_{\text{Sno}}(\text{Student})$ 表示“全部学生”。
- 除法结果是“被所有学生都选了的课程号”。
- 再与 Course 连接得到课程名。

关系代数表达式计算方法

计算表达式时按内到外做：

1. 先算括号里的关系。
2. 选择 σ ：删行。
3. 投影 π ：删列并去重。
4. 连接 $\bowtie$ ：先按条件配对，再保留需要的属性。
5. 差 $-$ ：从左关系中删除也在右关系中的元组。
6. 除 $\nabla \cdot$ ：对每个候选 X 看其象集是否包含 S 的所有 Y 。

小例子：
SC 中有：

Sno	Cno
S1	C1
S1	C2
S2	C1

Course 中有：

Cno
C1
C2

则：

$$\pi_{Sno,Cno}(SC) \nabla \cdot \pi_{Cno}(Course)$$

结果是 S1\$，因为\$S1\$同时有\$C1\$、\$C2\$；\$S2\$只有\$C1\$，不满足“所有课程”。

关系演算：与关系代数的区别

关系代数偏“过程”：说明怎么通过选择、投影、连接等运算得到结果。

关系演算偏“描述”：说明结果元组应满足什么谓词条件。

分类：

- 元组关系演算：以元组变量为基本对象。
- 域关系演算：以域变量为基本对象。

元组关系演算

一般形式：

$$\{t \mid P(t)\}$$

含义：所有使谓词 $P(t)$ 为真的元组 t 。

常见原子公式：

- $R(t)$ ：元组 t 属于关系 R 。
- $s[i] \theta t[j]$ ：元组 s 的第 i 个分量与元组 t 的第 j 个分量满足比较关系。
- $s[i] \theta c$ ：元组分量与常量比较。

常用符号：

- $\exists$ ：存在。
- $\forall$ ：任意/所有。
- $\wedge$ ：并且。

- $\vee$: 或者。
- $\neg$: 非。

常用关系运算在元组关系演算中的等价表示:

选择

$$\sigma_F(R) \equiv \{t \mid R(t) \wedge F(t)\}$$

含义: 先让元组 t 属于关系 R , 再加上筛选条件 $F(t)$ 。

投影

$$\pi_{i_1, i_2, \dots, i_k}(R) \equiv \{t \mid \exists u (R(u) \wedge t[1] = u[i_1] \wedge t[2] = u[i_2] \wedge \dots \wedge t[k] = u[i_k])\}$$

含义: 结果元组 t 的各分量, 分别取自原元组 u 的指定列。

并

$$R \cup S \equiv \{t \mid R(t) \vee S(t)\}$$

差

$$R - S \equiv \{t \mid R(t) \wedge \neg S(t)\}$$

交

$$R \cap S \equiv \{t \mid R(t) \wedge S(t)\}$$

5)在元组关系演算公式中, 各类算符优先级为:

- ①算术比较运算符优先级最高;
- ②量词 $\exists$ 和 $\forall$ 优先级其次, 且 $\exists$ 的优先级高于 $\forall$;
- ③逻辑运算符优先级最低, 且 $\neg$ 的优先级高于 $\wedge$, $\wedge$ 的优先级高于 $\vee$;
- ④括号可以改变算符的优先级, 加括号时, 括号内算符优先, 同一括号内算符优先级遵循上述各项。

例 1: 查询课程名为“数据结构”的学生学号和姓名。

Student(Sno, Sname, Sex, Age)
Course(Cno, Cname, Teacher)
SC(Sno, Cno, Grade)

$$\left\{ t \mid \begin{array}{l} \exists u \exists v \exists w (\text{Student}(u) \wedge \text{SC}(v) \wedge \text{Course}(w) \\ \wedge u[1] = v[1] \wedge v[2] = w[1] \wedge w[2] = \text{数据结构} \\ \wedge t[1] = u[1] \wedge t[2] = u[2]) \end{array} \right\}$$

理解：

- u 是学生元组， v 是选课元组， w 是课程元组。
- $u[1] = v[1]$ 把学生和选课按学号连接。
- $v[2] = w[1]$ 把选课和课程按课程号连接。
- $w[2] = \text{数据结构}$ 限定课程名。
- 最后用 $t[1]$ 、 $t[2]$ 指定输出列。

例 2：查询选修课程号为 C2 或 C4 的学生学号。

$$\{ t \mid \exists u (\text{SC}(u) \wedge (u[2] = \text{C2} \vee u[2] = \text{C4}) \wedge t[1] = u[1]) \}$$

例 3：查询至少同时选修 C2 和 C4 的学生学号。

$$\left\{ t \mid \begin{array}{l} \exists u \exists v (\text{SC}(u) \wedge \text{SC}(v) \wedge u[2] = \text{C2} \\ \wedge v[2] = \text{C4} \wedge u[1] = v[1] \wedge t[1] = u[1]) \end{array} \right\}$$

这个例子和关系代数中的 $\text{SC} \times \text{SC}$ 自连接对应。



元组关系演算

A	B	C
1	2	3
4	5	6
7	8	9

R

D	E
3	4
7	6

S

A	B	C
4	5	6
7	8	9

A	B	E
1	2	4
1	2	6
4	5	6

$$R1 = \{t | (\exists u)(R(t) \wedge S(u) \wedge t[3] > u[1])\}$$

$$R2 = \{t | (\exists u)(\exists v)(R(u) \wedge S(v) \wedge u[2] < v[1] \wedge t[1] = u[1] \wedge t[2] = u[2] \wedge t[3] = v[2])\}$$

选出来的元组还是要满足括号里的约束，所以上图中 R_2 不含元组 $(4, 5, 4)$ 。

域关系演算

一般形式：

$$\{x_1, x_2, \dots, x_k \mid P(x_1, x_2, \dots, x_k)\}$$

域演算以单个属性值为变量，而不是以整条元组为变量。

例：检索学习课程号为 C2 的学生学号与成绩。

完整写法：

$$\{t_1, t_2 \mid \exists u_1 \exists u_2 \exists u_3 (\text{SC}(u_1, u_2, u_3) \wedge u_2 = \text{C2} \wedge t_1 = u_1 \wedge t_2 = u_3)\}$$

可简化为：

$$\{t_1, t_2 \mid \text{SC}(t_1, \text{C2}, t_2)\}$$

元组演算和域演算的区别：

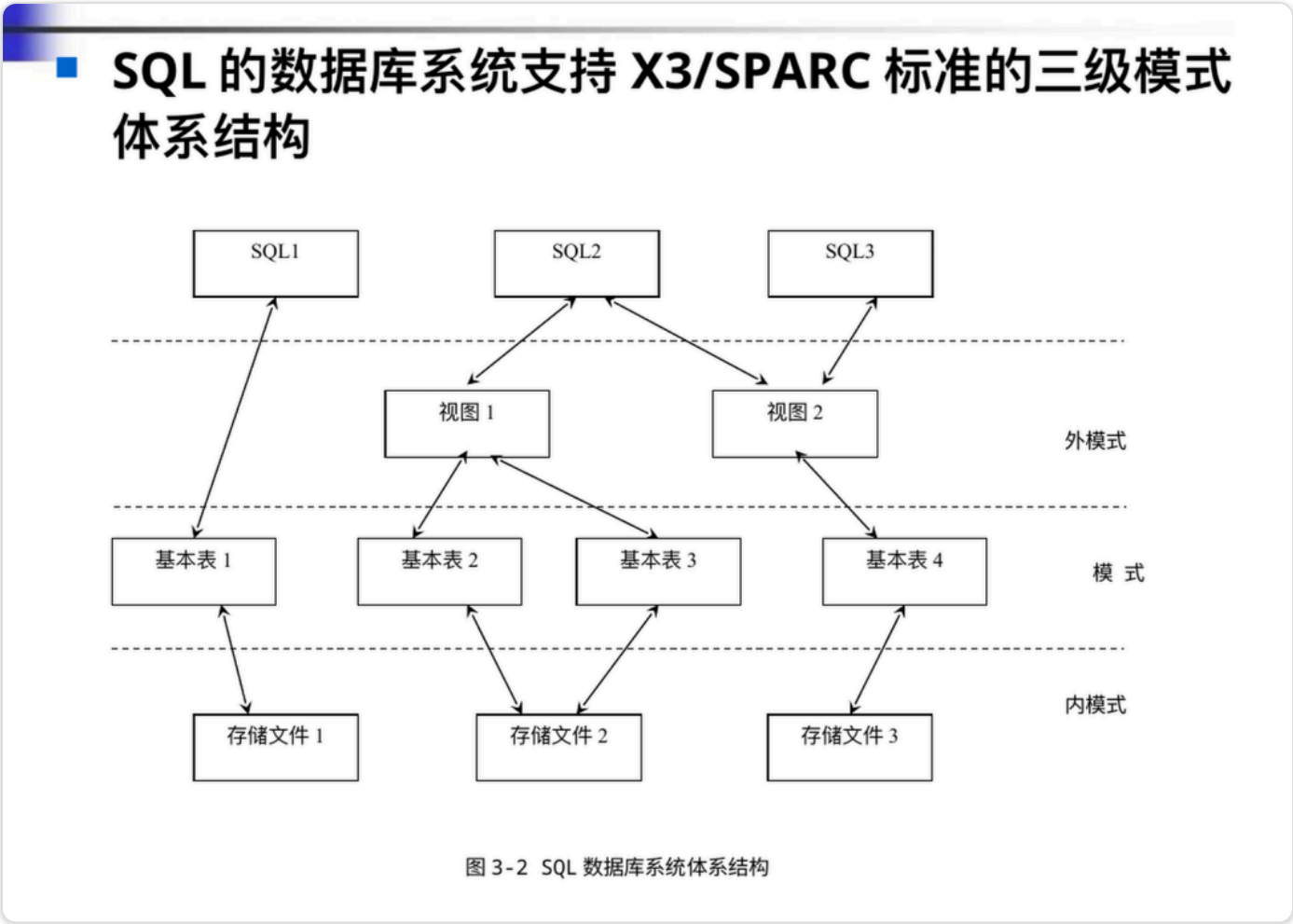
- 元组演算变量代表一整行，所以用 $u[1]$ 、 $u[2]$ 取分量。
 - 域演算变量代表某个属性值，所以直接写在关系谓词的位置上。
-

第三章 关系数据库语言 SQL

SQL 基本定位

SQL 是结构化查询语言，介于关系代数和关系演算之间，是通用的关系数据库语言。

组成结构



总记忆		
对象	是什么	记忆
模式 <i>Schema</i>	数据库对象的命名空间	装对象的逻辑容器
表 <i>Table</i>	真正存数据的关系	数据本体
视图 <i>View</i>	查询定义出的虚表	不一定存数据
索引 <i>Index</i>	加速查询的辅助结构	查找目录

SQL 的数据定义功能：模式定义、表定义、视图和索引的定义

表 3.2 SQL 的数据定义语句

操 作 对 象	操 作 方 式		
	创 建	删 除	修 改
模 式	CREATE SCHEMA	DROP SCHEMA	
表	CREATE TABLE	DROP TABLE	ALTER TABLE
视 图	CREATE VIEW	DROP VIEW	
索 引	CREATE INDEX	DROP INDEX	

核心功能

- 查询：SELECT
- 数据定义：CREATE、DROP、ALTER
- 数据操纵：INSERT、UPDATE、DELETE
- 数据控制：GRANT、REVOKE

- 数据操纵是改表的数据
- 数据控制是控制谁能访问数据

考试说明说第三章只对 CRUD 有书写要求。这里把重点放在“增删改查”：

- Create：插入记录 INSERT
- Read：查询记录 SELECT
- Update：修改记录 UPDATE
- Delete：删除记录 DELETE

另外，课件中 CREATE TABLE 出现较多，作为建表补充掌握。

考试提醒：

- 通常要求一条 SQL 写到底，不写 WITH ... AS ... 这种 CTE。
- 如果要把中间结果当作临时表，可以直接写在 FROM 里。
- 嵌入式 SQL 只需要知道概念，不要求写完整程序语句。

建表语法补充：CREATE TABLE

CREATE TABLE 不只是定义表名和列名，还要定义完整性约束。完整性约束的作用是防止数据库中出现不合法数据。

基本格式：

```
CREATE TABLE 表名 (  
    列名 数据类型 [列级完整性约束],  
    列名 数据类型 [列级完整性约束],  
    [表级完整性约束]  
);
```

完整性约束主要分三类：

类型	作用	常见 SQL 写法
实体完整性	保证每个元组可唯一识别，主键不能为空且不能重复	PRIMARY KEY
参照完整性	保证外键值必须能在被参照表中找到，或为空	FOREIGN KEY ... REFERENCES ...
用户定义完整性	根据具体业务限制属性取值	NOT NULL 、 UNIQUE 、 CHECK 、 DEFAULT

列级约束和表级约束

列级约束写在某一列定义后面，只约束这一列。

```
Sname CHAR(20) NOT NULL, -- NULL/NO NULL指定是否可以取空值  
Sage INT UNIQUE, -- 指定当前列取值不可重复  
Cname CHAR(10) DEFAULT NULL -- DEFAULT <expr> 指定该列的缺省值是<expr>
```

表级约束写在所有列定义后面，可以约束一列，也可以约束多列。联合主键、联合外键通常必须写成表级约束。

```
PRIMARY KEY (Sno, Cno),  
FOREIGN KEY (Sno) REFERENCES Student  
ON DELETE RESTRICT/CASCADE/SET NULL  
CHECK (Sage >= 0)
```

- *RESTRICT*：只要子表还有记录引用父表记录，就不能删除父表记录；
- *CASCADE*：删除父表时也删除子表
- *SET NULL*：删除父表时把子表的外键值改成NULL

实体完整性

实体完整性主要靠主键保证。

```
PRIMARY KEY (Sno)
```

含义：

- Sno 不能为 NULL。
- Sno 不能重复。
- 每个学生能被学号唯一标识。

联合主键：

```
PRIMARY KEY (Sno, Cno)
```

含义：单独的 Sno 可以重复，单独的 Cno 也可以重复，但同一个 (Sno, Cno) 组合不能重复。选课表 SC 常用这种写法，表示同一学生不能重复选同一门课。

参照完整性

参照完整性主要靠外键保证。

```
FOREIGN KEY (Sno) REFERENCES Student(Sno)
```

含义：SC.Sno 的值必须已经存在于 Student.Sno 中，否则这条选课记录就是“没有学生的选课记录”，不合法。

删除或修改父表记录时，常见处理方式：

写法	含义
ON DELETE CASCADE	删除父表记录时，自动删除子表中引用它的记录
ON DELETE RESTRICT	如果子表中仍有引用，拒绝删除父表记录
ON DELETE SET NULL	删除父表记录时，把子表外键设为 NULL，前提是该列允许空值

例如：

```
FOREIGN KEY (Sno) REFERENCES Student(Sno) ON DELETE CASCADE
```

这里父表是 `Student`，子表是 `SC`。删除 `Student` 中某个学生时，会自动删除 `SC` 中该学生的选课记录；但删除 `SC` 中一条选课记录，不会影响 `Student` 中的学生。

用户定义完整性

用户定义完整性是根据业务规则限制取值。

```
Grade DEC(4,1) CHECK (Grade IS NULL OR Grade BETWEEN 0 AND 100)
```

含义：成绩可以为空；如果不为空，必须在 0 到 100 之间。

```
Ssex CHAR(2) CHECK (Ssex IN ('男', '女'))
```

含义：性别只能取指定值。

```
Sdept CHAR(20) DEFAULT 'CS'
```

含义：插入学生时如果没有给出系别，默认填 `CS`。

例子：

```
CREATE TABLE SC (  
    Sno CHAR(7) NOT NULL,  
    Cno CHAR(6) NOT NULL,  
    Grade DEC(4,1) DEFAULT NULL,  
    PRIMARY KEY (Sno, Cno),  
    FOREIGN KEY (Sno) REFERENCES Student(Sno) ON DELETE CASCADE,  
    FOREIGN KEY (Cno) REFERENCES Course(Cno) ON DELETE RESTRICT,  
    CHECK (Grade IS NULL OR Grade BETWEEN 0 AND 100)  
);
```

解释：

- (Sno, Cno) 是联合主键，表示同一学生同一课程只能出现一次。

- 删除学生时， `ON DELETE CASCADE` 表示该学生的选课记录级联删除。

删除主表中的一条记录时，数据库自动把依赖它的子表记录一起删除。这里主表指外键Sno所述的Student，而子表指本表。

- 删除课程时， `ON DELETE RESTRICT` 表示如果有选课记录引用该课程，则限制删除。
- 成绩可以为空，也可以是 0-100。

对其他数据库对象的CREATE看PPT

查询：SELECT

基本格式：

```
SELECT [ALL | DISTINCT] 目标列表达式  
FROM 表名或视图名  
[WHERE 条件表达式]  
[GROUP BY 列名 [HAVING 条件表达式]]  
[ORDER BY 列名 [ASC | DESC]];
```

执行理解：

1. `FROM`：确定数据来源。
2. `WHERE`：筛选行。
3. `GROUP BY`：分组。
4. `HAVING`：筛选组。
5. `SELECT`：确定输出列或表达式。

基本查询语句

- 整个 **SELECT** 语句的含义是，根据 **WHERE** 子句的条件表达式，从 **FROM** 子句指定的基本表或视图找出满足条件的元组，再按 **SELECT** 子句中的目标列表表达式，选出元组中的属性值形成结果表
- 如果有 **GROUP** 子句，则将结果按 <列名 1> 的值进行分组，该属性列值相等的元组为一个组，每个组产生结果表中的一条记录，通常会在每组中作用集函数
- 如果 **GROUP** 子句带 **HAVING** 短语，则只有满足指定条件的组才予输出
- 如果有 **ORDER** 子句，则结果表还要按 <列名 2> 的值的升序或降序进行排序

- 一个典型的 SQL 查询采用如下的基本形式：

```
SELECT   $A_1, A_2, \dots, A_n$   
FROM     $r_1, r_2, \dots, r_m$   
WHERE   $P$ 
```

- 其等价的关系代数表达式：

$$\Pi_{A_1, A_2, \dots, A_n} (\sigma_P(r_1 \times r_2 \times \dots \times r_m))$$

选择列

```
SELECT Sno, Sname  
FROM Student;
```

查询全体学生的学号和姓名。

查询全部列：

```
SELECT *  
FROM Student;
```

去重 DISTINCT

```
SELECT DISTINCT Sno  
FROM SC;
```

查询选修过课程的学生学号，并去重。

如果不加 `DISTINCT`，默认是 `SELECT ALL`，不去重。

WHERE 条件

比较：

```
SELECT Sname, Sage  
FROM Student  
WHERE Sage < 20;
```

补充几个运算符：

`<>` `!>` `!<`，分别代表不等于，不大于，不小于

范围：

```
SELECT Sname, Sdept, Sage
FROM Student
WHERE Sage BETWEEN 20 AND 23; -- 闭区间，两边都能取到。
-- or NOT BETWEEN 20 AND 23
```

集合：

```
SELECT Sname, Ssex
FROM Student
WHERE Sdept IN ('IS', 'MA', 'CS');
-- Sdept 就是 学生所在系，stu department
```

字符匹配：

```
SELECT Sname, Sno, Ssex
FROM Student
WHERE Sname LIKE '刘%';
```

LIKE 中：

- %：匹配任意长度字符串。
- _：匹配单个字符。

```
SELECT Cno , Ccredit
FROM Course
WHERE Cname LIKE 'DB\__%i\_ ' ESCAPE '\ ' ;
```

查询以“DB_”开头，倒数第三个字符是i的课程的具体情况。 ESCAPE '\ ' 表示“\”是转义字符。
空值：

```
SELECT Sno, Cno
FROM SC
WHERE Grade IS NULL;
```

注意：判断空值必须用 IS NULL 或 IS NOT NULL ，不能写 Grade = NULL 。

多条件：

```
SELECT Sname
FROM Student
WHERE Sdept = 'CS' AND Sage < 20;
```

AND 优先级高于 **OR**，复杂条件最好加括号。

ORDER BY 排序

```
SELECT Sno, Grade
FROM SC
WHERE Cno = 'CS-03'
ORDER BY Grade DESC;
```

- **ASC**：升序，默认。
- **DESC**：降序。

如何处理 **NULL**？

- 升序的时候空值先显示；
- 降序的时候空值最后显示

多列排序：

```
SELECT *
FROM Student
ORDER BY Sdept ASC, Sage DESC;
```

先按系升序，同一系内按年龄降序。

函数

包括标量函数和聚集函数。

- **标量函数的运算对象是一个记录行在某个属性上的具体取值，大致可以分为字符、数学、时间、类型转换等几种形式。例如，函数 `LENGTH()` 可以计算一个字符串类型数值的长度，`ABS()` 可以计算一个数值类型的绝对值**

聚集函数

具有**统计性质**的一类函数，其运算对象通常是记录的集合或一组记录在某个列上的全部取值，聚集函数的返回结果一般将会是唯一的一个确定值。

常用聚集函数：

- `COUNT(*)`：统计行数。
- `COUNT(DISTINCT 列名)`：统计去重后的非空值数量。
- `SUM(列名)`：求和。
- `AVG(列名)`：平均值。
- `MAX(列名)`：最大值。
- `MIN(列名)`：最小值。

例子：

查询学生总人数。

```
SELECT COUNT(*)  
FROM Student;
```

查询 CS-01 课程平均成绩。

```
SELECT AVG(Grade)  
FROM SC  
WHERE Cno = 'CS-01';
```

查询选修CS-01课程的学生最高分数

```
SELECT MAX(Grade)
FROM SC
WHERE Cno = 'CS-01';
```

查询学号为200215012的学生的选修课程的总学分数

```
SELECT SUM(Ccredit)
FROM SC, Course
WHERE SC.Cno = Course.Cno AND Sno = '200215012'
```

GROUP BY 与 HAVING

GROUP BY

— 细化聚集函数的作用对象

— 未对查询结果分组，聚集函数将作用于整个查询结果

— 对查询结果分组后，聚集函数将分别作用于每个组

— 作用对象是查询的**中间结果表**

— 按指定的一列或多列值分组，值相等的为一组

所以下例中 `COUNT(Sno)` 对应的是分组后各个组的学生人数。

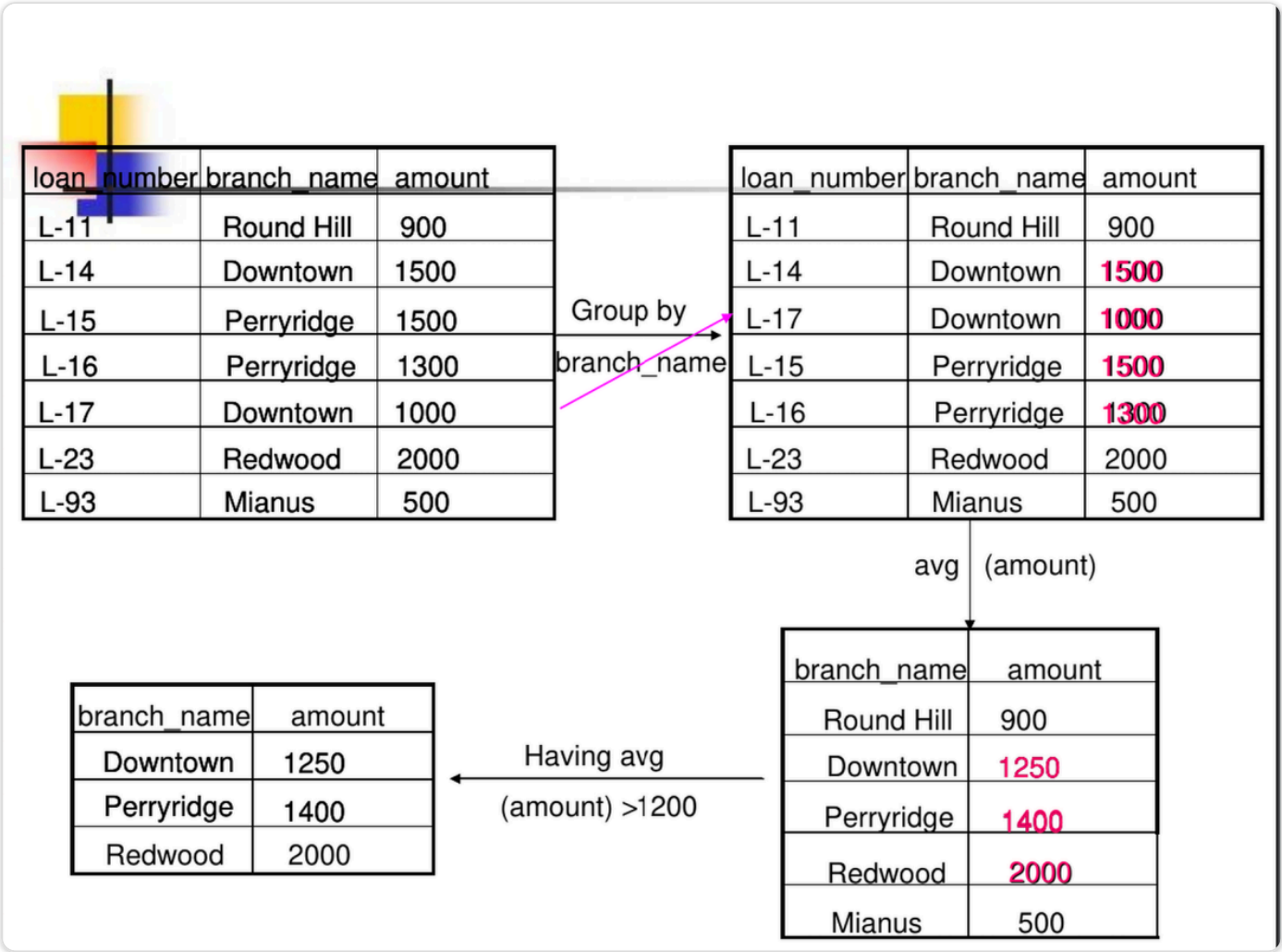
```
-- 查询每门课的选课人数。
SELECT Cno, COUNT(Sno)
FROM SC
GROUP BY Cno;
```

HAVING

作用于分组后的各个组，筛选符合要求的组。

```
-- 查询选修了 3 门及以上课程的学生学号。  
SELECT Sno  
FROM SC  
GROUP BY Sno  
HAVING COUNT(*) >= 3;
```

- WHERE 与 HAVING 的区别:
- WHERE 作用于分组前的行。
 - HAVING 作用于分组后的组。



对应SQL语句为:

```
SELECT branch_name, AVG(amount)
FROM loan
GROUP BY branch_name
HAVING AVG(amount) > 1200;
```

sno	cno	grade
801	01	92
801	02	78
801	03	85
802	01	82
802	02	90
803	01	88

平均成绩最高的学生学号及平均成绩？

```
SELECT Sno, AVG(Grade)
FROM SC
GROUP BY Sno
HAVING AVG(Grade) ≥ ALL ( -- 后面这个跟着嵌套查询
    SELECT AVG(Grade)
    FROM SC
    GROUP BY Sno
);
```

如果是输出这个学生的各科成绩？

```
SELECT SC.Sno, SC.Cno, SC.Grade
FROM SC,(
  SELECT Sno, AVG(Grade)
  FROM SC
  GROUP BY Sno
  HAVING AVG(Grade) ≥ ALL(
    SELECT AVG(Grade)
    FROM SC
    GROUP BY Sno
  )
) AS T
WHERE SC.Sno = T.Sno
```

连接查询

连接查询

- 等值与非等值连接查询
- 自连接
- 外连接
- 复合条件连接

等值连接与非等值

一言以蔽之：等值是保留重复列，重命名；自然连接是去掉重复列。

- 等值连接：连接运算符为 =
- 例 3-41：查询每个学生及其选修课程的情况

等值连接语法如下：

```
SELECT Student.* , SC.* (*)  
FROM Student , SC  
WHERE Student.Sno = SC.Sno ;
```

可以分别写 `.*`，也可以直接写`*`。这样会保留重复列。

而自然连接如下：

- **例 3-42：对 [例 3-41] 用自然连接完成**

```
SELECT Student.Sno , Sname , Ssex , Sage , Sdept ,  
Cno , Grade  
FROM Student , SC  
WHERE Student.Sno = SC.Sno ;
```

手动从两个表之一中指定重复属性。

自连接

需要给表起别名来表示区别。

由于所有属性名都是同名属性，因此必须使用别名前缀
例 3-43：查询每一门课的间接先修课（即先修课的先修课）

```
SELECT FIRST.Cno , SECOND.Cpno  
FROM Course FIRST , Course SECOND  
WHERE FIRST.Cpno = SECOND.Cno ;
```

外连接



外连接

- 外连接与普通连接的区别
 - 普通连接操作只输出满足连接条件的元组
 - 外连接操作以指定表为连接主体，将主体表中不满足连接条件的元组一并输出
- 例 3-44：改写 [例 3-42]

```
SELECT Student.Sno , Sname , Ssex , Sage , Sdept , Cno ,  
Grade  
FROM Student LEFT OUTER JOIN SC ON (Student.Sno=SC.Sno) ;
```


复合条件连接

查询选修了“数据库系统原理”的学生学号和姓名：

```
SELECT Student.Sno, Student.Sname
FROM Student, SC, Course
WHERE Student.Sno = SC.Sno
      AND SC.Cno = Course.Cno
      AND Course.Cname = '数据库系统原理';
```

理解：三表连接时，必须写连接条件，否则会形成笛卡尔积，结果大量错误。

嵌套查询（看PPT例题！）

• 嵌套查询概述

- 一个 **SELECT-FROM-WHERE** 语句称为一个**查询块**
- 将一个查询块嵌套在另一个查询块的 **WHERE** 子句或 **HAVING** 短语的条件中的查询称为**嵌套查询**

查询选修了课程号 C2 的学生姓名：

```
SELECT Sname
FROM Student
WHERE Sno IN (
  SELECT Sno
  FROM SC
  WHERE Cno = 'C2'
);
```

查询没有选修 C2 的学生姓名：

```
SELECT Sname
FROM Student
WHERE Sno NOT IN (
    SELECT Sno
    FROM SC
    WHERE Cno = 'C2'
);
```

更稳的“未选”写法可用 `NOT EXISTS`：

```
SELECT Sname
FROM Student
WHERE NOT EXISTS (
    SELECT *
    FROM SC
    WHERE SC.Sno = Student.Sno
    AND SC.Cno = 'C2'
);
```

`FROM` 中的派生表

有些题目需要先算一个中间结果，再拿它继续查询。考试里通常不用 `WITH`，可以直接把子查询放进 `FROM`，这就是派生表，也可以理解成临时表。

例如：查询平均成绩高于 80 的学生学号和平均成绩。

```
SELECT T.Sno, T.AvgGrade
FROM (
    SELECT Sno, AVG(Grade) AS AvgGrade
    FROM SC
    GROUP BY Sno
    HAVING AVG(Grade) > 80
) AS T;
```

这类写法的要点是：

- 子查询放在 `FROM` 里，外层把它当成一张临时表。
- 派生表必须起别名，比如这里的 `T`。
- 考试里如果题目能一条 SQL 直接写出来，优先直接写，不要绕成多层 CTE。

谓词

有时间看一下PPT。

- **集合操作的种类**
 - 并操作 **UNION**
 - 交操作 **INTERSECT**
 - 差操作 **EXCEPT**
- **参加集合操作的各查询结果的列数必须相同；对应项的数据类型也必须相同**

看PPT。

插入：INSERT

单记录插入：

```
INSERT INTO Course (Cno, Cname, Teacher)
VALUES ('CS-06', '数据库系统原理', '李华');
```

如果只插入部分列，未列出的列会取默认值或 **NULL**，前提是约束允许。

```
INSERT INTO SC (Sno, Cno)
VALUES ('01055123', 'CS-06');
```

这里 **Grade** 未给出，如果定义允许空值，则成绩暂缺。

子查询结果插入：

```
INSERT INTO S_Grade (Sno, Avg_Grade)
SELECT S.Sno, AVG(SC.Grade)
FROM Student AS S, SC
WHERE S.Sno = SC.Sno
AND S.Ssex = '男'
GROUP BY S.Sno
HAVING AVG(SC.Grade) > 80;
```

含义：把平均成绩高于 80 分的男生学号和平均成绩插入 `S_Grade`。

删除：DELETE

基本格式：

```
DELETE FROM 表名  
[WHERE 条件表达式];
```

删除某个学生：

```
DELETE FROM Student  
WHERE Sno = '01055123';
```

删除“C语言”课程中成绩为空的选课记录：

```
DELETE FROM SC  
WHERE Grade IS NULL  
AND Cno IN (  
    SELECT Cno  
    FROM Course  
    WHERE Cname = 'C语言'  
);
```

重要易错点：没有 `WHERE` 会删除整张表的全部记录。

```
DELETE FROM Course;
```

这不是删除表结构，而是删除 `Course` 表中的全部记录。删除表结构是 `DROP TABLE Course;`。

修改：UPDATE

基本格式：

```
UPDATE 表名  
SET 列名 = 表达式 [, 列名 = 表达式]  
[WHERE 条件表达式];
```

给所有员工工资加 100:

```
UPDATE EMP
SET Salary = Salary + 100;
```

把某学生某门课成绩置为空:

```
UPDATE SC
SET Grade = NULL
WHERE Sno = '01055071'
AND Cno = 'CS-03';
```

重要易错点: 没有 `WHERE` 会修改整张表所有记录。

SQL 书写题常见检查清单

写完 SQL 后按这几个问题检查:

1. 输出列是否正好是题目要求的列?
2. 多表查询是否写了连接条件?
3. 是否需要去重 `DISTINCT` ?
4. 空值判断是否用了 `IS NULL` , 而不是 `= NULL` ?
5. 聚集函数是否需要 `GROUP BY` ?
6. 分组后筛选是否用了 `HAVING` ?
7. `UPDATE` 和 `DELETE` 是否遗漏 `WHERE` ?
8. 子查询返回多值时是否用了 `IN` , 而不是 `=` ?
9. “全部/所有”类 SQL 题是否适合用双重 `NOT EXISTS` ?

例: 查询选修了全部课程的学生姓名。

```
SELECT Sname
FROM Student
WHERE NOT EXISTS (
    SELECT *
    FROM Course
    WHERE NOT EXISTS (
        SELECT *
        FROM SC
        WHERE SC.Sno = Student.Sno
        AND SC.Cno = Course.Cno
    )
);
```

理解: 不存在这样一门课程, 使得该学生没有选。也就是该学生选了所有课程。

嵌入式 SQL（了解概念）

嵌入式 SQL 是把 SQL 语句嵌入到高级语言程序中使用，例如 C、COBOL、FORTRAN。
为什么需要嵌入式 SQL：

- SQL 擅长集合操作。
- 高级语言擅长过程控制、复杂业务逻辑、界面和输入输出。
- 嵌入式 SQL 用高级语言补足 SQL 的过程控制能力。

基本形式：

```
EXEC SQL <SQL语句>;
```

核心概念：

概念	含义
主语言/宿主语言	嵌入 SQL 的高级语言，如 C
EXEC SQL	区分 SQL 语句与主语言语句的前缀
预处理	先把嵌入式 SQL 转换成主语言函数调用，再交给主语言编译器
SQLCA	SQL 通信区，保存 SQL 执行状态信息
SQLCODE	SQLCA 中的返回代码，用来判断执行成功或失败
主变量	SQL 中引用的宿主语言变量，SQL 语句中前面加冒号
指示变量	用于表示主变量是否为空、是否被截断等状态
游标	处理多行查询结果的缓冲区和指针机制

主变量例子：

```
EXEC SQL SELECT Sno, Sname
        INTO :Hsno, :Hname
        FROM Student
        WHERE Sno = :givensno;
```

解释：

- :givensno 是输入主变量，由程序提供查询条件。
- :Hsno 、 :Hname 是输出主变量，用来接收查询结果。

游标使用步骤：

1. **DECLARE CURSOR**：声明游标，对应一条 **SELECT**。
2. **OPEN**：打开游标，执行查询，把结果放入缓冲区。
3. **FETCH**：逐条取结果到主变量。
4. **CLOSE**：关闭游标，释放资源。

游标模板：

```
EXEC SQL DECLARE C CURSOR FOR
  SELECT Cno, Grade
  FROM SC
  WHERE Sno = :Hsno;

EXEC SQL OPEN C;

EXEC SQL FETCH C INTO :Hcno, :Hgrade;

EXEC SQL CLOSE C;
```

什么时候不需要游标：

- 数据定义语句，如 **CREATE TABLE**、**DROP TABLE**。
- 数据控制语句，如授权、收权。
- 结果为单记录的 **SELECT INTO**。
- 普通 **INSERT**、**UPDATE**、**DELETE**。

什么时候需要游标：

- **SELECT** 查询结果可能有多条记录，而宿主语言一次只能处理一条记录。
-

第四章 数据依赖与关系模式规范化

考试注意事项：第 4 章算法多，要熟练掌握，但不要求证明。重点是属性集闭包、候选键、无损连接、3NF/BCNF 分解合成、函数依赖闭包。

函数依赖基本概念

函数依赖写作：

$$X \rightarrow Y$$

含义：在关系模式

$R(U)$ 中，任意两个元组如果在属性集 X 上取值相同，则在属性集 Y 上取值也必须相同。此时称 X 函数决定 Y 。

常见类型：

- 平凡函数依赖：若 $Y \subseteq X$ ，则 $X \rightarrow Y$ 是平凡依赖。
- 非平凡函数依赖：若 $Y \not\subseteq X$ ，则 $X \rightarrow Y$ 是非平凡依赖。
- 完全函数依赖： $X \rightarrow Y$ 成立，但任意真子集 $X' \subset X$ 都不能推出 Y 。
- 部分函数依赖： $X \rightarrow Y$ 成立，且存在真子集 $X' \subset X$ ，使 $X' \rightarrow Y$ 成立。
- 传递函数依赖：若 $X \rightarrow Y$ ， $Y \rightarrow Z$ ，且 $Y \nrightarrow X$ ，则 Z 传递依赖于 X 。

例子：

$$(Sno, Cno) \rightarrow Grade$$

表示一个学生选一门课后成绩唯一确定。

$$Cno \rightarrow Teacher$$

表示课程号唯一决定任课教师。如果在关系

$$R(Sno, Cno, Grade, Teacher)$$

中候选键是 (Sno, Cno) ，那么 $Teacher$ 对候选键存在部分依赖，因为只靠 Cno 就能决定它。

属性集闭包

属性集 X 关于函数依赖集 F 的闭包记为：

$$X_F^+ = \{A \mid A \in U, F \models X \rightarrow A\}$$

直观理解：从 X 出发，利用 F 中的函数依赖，最多能推出哪些属性。

算法步骤：

- 初始令 $X^+ = X$ 。
- 扫描函数依赖集 F ，若存在 $Y \rightarrow Z$ 且 $Y \subseteq X^+$ ，则把 Z 加入 X^+ 。
- 重复第 2 步，直到 X^+ 不再变化。
- 若最终 $X^+ = U$ ，则 X 是超键。

例子：

$$U = \{A, B, C, D, E\}$$

$$F = \{AB \rightarrow C, B \rightarrow D, C \rightarrow E, EC \rightarrow B, AC \rightarrow B\}$$

求 $(AB)_F^+$:

$$X^+ = AB$$

由 $AB \rightarrow C$ 、 $B \rightarrow D$:

$$X^+ = ABCD$$

由 $C \rightarrow E$:

$$X^+ = ABCDE$$

所以:

$$(AB)_F^+ = ABCDE$$

用途:

- 判断 $X \rightarrow Y$ 是否被 F 蕴含: 只要看 $Y \subseteq X_F^+$ 。
- 判断 X 是否为超键: 只要看 $X_F^+ = U$ 。
- 求候选键、判断无损连接和依赖保持时都要反复用闭包。

函数依赖集闭包与等价

函数依赖集 F 的闭包记为:

$$F^+$$

它表示由 F 能推出的所有函数依赖。

两个函数依赖集 F 和 G 等价, 当且仅当:

$$F^+ = G^+$$

实际判断时不用真的求完整闭包, 而是互相检查覆盖:

$$F \subseteq G^+ \quad \text{且} \quad G \subseteq F^+$$

做题方法:

- 对 F 中每个 $X \rightarrow Y$, 用 G 求 X_G^+ , 看 Y 是否包含在其中。
- 对 G 中每个 $X \rightarrow Y$, 用 F 求 X_F^+ , 看 Y 是否包含在其中。

最小函数依赖集

最小覆盖也叫最小函数依赖集。它通常满足:

1. 每个函数依赖右部只有一个属性。
2. 每个函数依赖左部没有多余属性。
3. 没有多余的函数依赖。

求最小覆盖步骤:

1. 拆右部:

$$X \rightarrow A_1 A_2 \cdots A_k$$

改成：

$$X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_k$$

2. 消左部冗余属性：对每个 $X \rightarrow A$ ，若 $B \in X$ 且

$$A \in (X - \{B\})_F^+$$

则 B 是多余属性，可以从左部删去。

3. 删除冗余依赖：对每个 $X \rightarrow A$ ，令

$$G = F - \{X \rightarrow A\}$$

若：

$$A \in X_G^+$$

则 $X \rightarrow A$ 是冗余依赖，可以删除。

注意：最小覆盖不一定唯一，和处理依赖、处理属性的顺序有关。

求候选键

候选键是最小超键：能推出全部属性，且去掉任意属性后不能推出全部属性。

课件给出的常用分类方法：

- C_1 ：不在任何函数依赖中出现的属性。
- C_2 ：只在函数依赖左部出现的属性。
- C_3 ：只在函数依赖右部出现的属性。
- C_4 ：左右部都出现的属性。

关键结论：

$$C_1 \cup C_2$$

一定包含在任何候选键中。

算法步骤：

1. 先计算：

$$(C_1 \cup C_2)^+$$

若等于 U ，则 $C_1 \cup C_2$ 是唯一候选键。

2. 若不等于 U ，从 C_4 中逐个加入属性，计算闭包。
3. 找到能推出 U 的属性集，再检查是否最小。
4. 若加入一个属性不够，就尝试加入两个、三个，直到找全所有候选键。

例子：

$$R(A, B, C, D), \quad F = \{A \rightarrow B, B \rightarrow C, BD \rightarrow A\}$$

分类：

$$C_1 = \emptyset, \quad C_2 = \{D\}, \quad C_3 = \{C\}, \quad C_4 = \{A, B\}$$

先看：

$$D^+ = D$$

不是超键。加入 A ：

$$(AD)^+ = ADBC = U$$

加入 B ：

$$(BD)^+ = BDAC = U$$

所以候选键为：

$$AD, BD$$

范式判断

常用范式关系：

$$BCNF \Rightarrow 3NF \Rightarrow 2NF \Rightarrow 1NF$$

1NF

每个属性值都是不可再分的原子值。

如果一个属性里存了“多个电话”“多个课程列表”，就不满足 1NF。

2NF

在 1NF 基础上，所有非主属性都完全函数依赖于候选键。

违反 2NF 的典型情况：组合键的一部分就能决定某个非主属性。

例子：

$$R(\text{Sno}, \text{Cno}, \text{Grade}, \text{Teacher})$$

候选键是 (Sno, Cno) ，但：

$$\text{Cno} \rightarrow \text{Teacher}$$

所以 Teacher 部分依赖于候选键， R 不满足 2NF。

3NF

在 2NF 基础上，消除非主属性对候选键的传递依赖。

等价判定条件：对每个非平凡函数依赖 $X \rightarrow A$ ，至少满足下列之一：

1. X 是超键。
2. A 是主属性。

例子：

$$\text{Cno} \rightarrow \text{Teacher}, \quad \text{Teacher} \rightarrow \text{Dept}$$

则：

$$\text{Cno} \rightarrow \text{Dept}$$

是传递依赖，通常违反 3NF。

BCNF

对每个非平凡函数依赖：

$$X \rightarrow Y$$

都要求 X 必须是超键。

BCNF 比 3NF 更严格。3NF 允许右部是主属性的某些情况，但 BCNF 不允许“决定子不是超键”。

范式判断套路：

1. 先求所有候选键。
2. 标出主属性和非主属性。
3. 看是否 1NF。
4. 看是否存在非主属性对候选键的部分依赖，判断 2NF。
5. 看是否存在非主属性对候选键的传递依赖，判断 3NF。
6. 看每个非平凡依赖的左部是否都是超键，判断 BCNF。

无损连接分解

分解：

$$\rho = \{R_1, R_2, \dots, R_k\}$$

是无损连接分解，直观意思是：原关系投影到各个子模式后，再自然连接回来，不产生错误元组，也不丢信息。

二元分解快速判定

若：

$$\rho = \{R_1, R_2\}$$

则无损连接的充要条件是：

$$(R_1 \cap R_2) \rightarrow (R_1 - R_2)$$

或：

$$(R_1 \cap R_2) \rightarrow (R_2 - R_1)$$

例子：

$$R(A, B, C), \quad \rho = \{AB, BC\}$$

公共属性是：

$$AB \cap BC = B$$

若 F 中能推出：

$$B \rightarrow A$$

或：

$$B \rightarrow C$$

则该二元分解无损。

多元分解表格法

用于 $\rho = \{R_1, \dots, R_k\}$ 。

步骤：

1. 建一个 k 行、 n 列的表。列对应 R 的全部属性 $A_1, \dots, A_n$ ，行对应每个子模式 R_i 。
2. 如果属性 $A_j \in R_i$ ，该格填 a_j ；否则填 b_{ij} 。
3. 反复使用函数依赖 $X \rightarrow Y$ ：若若干行在 X 属性列上的符号相同，就把这些行在 Y 属性列上的符号改成一致。
4. 如果某一行变成：

$$a_1, a_2, \dots, a_n$$

则分解无损。

5. 若表不再变化且没有全 a 行，则不是无损连接分解。

保持函数依赖

分解 $\rho = \{R_1, \dots, R_k\}$ 保持函数依赖，意思是：原来的依赖 F 能由各子模式上的依赖投影联合推出。

形式上：

$$\left(\bigcup_i \pi_{R_i}(F) \right)^+ \supseteq F$$

做题判断方法：

1. 对 F 中每个依赖 $X \rightarrow Y$ 逐个检查。
2. 令 $Z = X$ 。
3. 对每个子模式 R_i ，计算：

$$(Z \cap R_i)^+ \cap R_i$$

并加入 Z 。

4. 反复直到 Z 不再变化。
5. 若 $Y \subseteq Z$ ，则该依赖被保持；所有依赖都被保持，则分解保持函数依赖。

3NF 分解合成算法

目标：得到既保持函数依赖，又无损连接的 3NF 模式集。

步骤：

1. 求 F 的最小覆盖 $F_{\min}$ 。
2. 对 $F_{\min}$ 中左部相同的依赖分组。

3. 每组生成一个关系模式，属性为该组依赖中出现的全部属性。
4. 如果某个生成的关系模式是另一个的子集，可以删去较小者。
5. 若没有任何一个关系模式包含原关系的候选键，则额外加入一个候选键关系模式。

例子模板：

若有：

$$F_{\min} = \{A \rightarrow B, A \rightarrow C, D \rightarrow E\}$$

按左部分组：

$$A \rightarrow BC, \quad D \rightarrow E$$

生成：

$$R_1(A, B, C), \quad R_2(D, E)$$

如果这两个都不含候选键，还要加一个候选键模式 R_{ck} 。

BCNF 分解算法

目标：得到无损连接的 BCNF 模式集，但不一定保持函数依赖。

步骤：

1. 初始：

$$\rho = \{R(U)\}$$

2. 若 ρ 中每个子模式都是 BCNF，结束。
3. 若某个子模式 R_i 不满足 BCNF，则存在非平凡依赖：

$$X \rightarrow A$$

且 X 不是 R_i 的超键。

4. 将 R_i 分解为：

$$R_{i1} = X \cup A$$

$$R_{i2} = R_i - \{A\}$$

5. 用 R_{i1}, R_{i2} 替换 R_i ，继续检查。

考试易错点：BCNF 分解一般保证无损连接，但可能不保持函数依赖。若要求“无损且保持依赖”，通常优先用 3NF 合成算法。

第五章 数据库设计

考试注意事项：先画 ER 图再写关系模式；不考虑弱实体；联系和实体最好体现属性； $1:1$ 和 $1:N$ 不考虑部分参与和全参与；只有 $M:N$ 的二元联系才转化成额外关系模式；不要求会画扩展 ER 图。
本章按“考试版规则”复习即可：普通实体、普通属性、普通二元联系是重点；弱实体、扩展 ER、部分参与/全参与不作为本次考试处理对象。

ER 模型基本元素

ER 模型主要包括：

- 实体：现实世界中可区分的对象，如学生、课程、教师。
- 属性：实体或联系的特征，如学号、姓名、成绩。
- 联系：实体之间的关联，如学生选修课程。

画 ER 图时要注意：

- 实体要有属性，最好标出键。
- 联系如果自身有属性，也要画出来。例如“选课”联系有成绩。
- 联系要标明基数约束： $1:1$ 、 $1:N$ 、 $M:N$ 。

ER 设计步骤

做题建议：

1. 读题找名词，先确定实体。
2. 找动词或业务事件，确定联系。
3. 给实体补属性，给联系补联系属性。
4. 标基数：一对一、一对多、多对多。
5. 再转换成关系模式。

例子：

“学生选修课程，并记录成绩。”

实体：

学生(学号, 姓名, 性别)

课程(课程号, 课程名, 学分)

联系：

选修(成绩)

基数：

学生 $M:N$ 课程

ER 图转关系模式规则

实体转换

每个实体转换为一个关系模式：

$$E(K, A_1, A_2, \dots, A_n)$$

其中实体的键 K 成为关系模式的主键。

例子：

学生(学号, 姓名, 性别)

1:1 二元联系

按本次考试注意事项，1:1 联系不考虑部分参与和全参与，也不额外建立联系表。处理方法：把其中一端的主键加入另一端作为外键，并把联系自身的属性一起放进去。

例子：

学校 1:1 校长

可合并为：

学校(校名, 地址, 校长编号, 任职年月)

1:N 二元联系

按本次考试注意事项，1:N 联系通常合并到 N 端：把 1 端主键放入 N 端作为外键，并加入联系属性，不额外建联系表。

例子：

系 1:N 教师

若联系“聘用”有属性“聘期”，则：

系(系号, 系名, 电话)

教师(教师号, 姓名, 性别, 系号, 聘期)

其中 系号 是外键。

M:N 二元联系

这是考试注意事项中特别强调的规则：只有多对多的二元联系才必须转化成额外关系模式。

转换方法：两个实体各自成表，联系也成表；联系表包含双方主键和联系属性。

例子：

学生 $M:N$ 课程

转换为：

学生(学号, 姓名, 性别)

课程(课程号, 课程名, 学分)

选课(学号, 课程号, 成绩)

其中:

- 学号 是引用学生的外键。
- 课程号 是引用课程的外键。
- (学号, 课程号) 联合构成选课表主键。

本次不展开的情况

本次提纲明确“不考虑弱实体”“不要求扩展 ER 图”。因此复习时不要把精力放在弱实体、子类/超类、聚集、范畴等扩展 ER 内容上。

三元联系也不在这版提纲的重点里。若题目没有明确给三元联系，按普通二元联系规则处理即可；对二元联系，额外建联系表只用于 $M:N$ 。

多值属性和复合属性

- 复合属性：可以横向展开。例如地址可拆成省、市、街道。
- 多值属性：通常纵向展开，单独建一个关系模式。

例子：学生有多个电话号码。

不要写成：

学生(学号, 姓名, 电话列表)

更合适：

学生(学号, 姓名)

学生电话(学号, 电话)

ER 转关系模式答题模板

题目要求“先画 ER 图再写关系模式”时，建议按这个顺序写：

1. 实体：

实体名(主键, 属性₁, 属性₂, ...)

2. 联系：

实体₁ 1 : N 实体₂

或：

实体₁ $M : N$ 实体₂

3. 转换结果：

- 实体各自转换。
- 1 : 1 联系并入任一端。
- 1 : N 联系并入 N 端。

- $M : N$ 联系生成额外关系模式。
 - 不写弱实体、扩展 ER、全参与/部分参与相关规则。
-

第七章 查询处理与查询优化

考试注意事项：代数优化/语法优化、语法树、笛卡尔积、选择连接、物理优化使用时机。

查询处理基本流程

SQL 查询一般经过：

1. 词法分析、语法分析。
2. 生成内部表示，通常是语法树或查询树。
3. 代数优化：用关系代数等价变换改写表达式。
4. 物理优化：选择具体存取路径和操作算法。
5. 生成执行计划并执行。

语法树中常见对应关系：

- **FROM**：形成关系来源，朴素形式可看成笛卡尔积。
- **WHERE**：选择条件，对应 σ 。
- **SELECT**：输出列，对应 π 。

语法优化和代数优化的关系：

- 语法优化：从 SQL 语句或语法树出发，识别 **SELECT-FROM-WHERE** 的结构，把它转换成关系代数表达式或查询树，并尽量避免明显低效的语法结构。
- 代数优化：在已经得到的关系代数表达式上，用等价变换规则进行改写，例如选择下推、投影下推、把笛卡尔积加选择改成连接。
- 物理优化：不再只是改写表达式，而是选择具体执行方法，例如用索引扫描、排序归并连接、散列连接等。

例子：

```
SELECT Sname
FROM Student, SC
WHERE Student.Sno = SC.Sno
      AND Cno = 'C2';
```

朴素关系代数可写成：

$$\pi_{\text{Sname}}(\sigma_{\text{Student.Sno}=\text{SC.Sno} \wedge \text{Cno}=\text{C2}}(\text{Student} \times \text{SC}))$$

优化后应把选择和连接提前：

$$\pi_{\text{Sname}}(\text{Student} \bowtie_{\text{Student.Sno}=\text{SC.Sno}} \sigma_{\text{Cno}=\text{C2}}(\text{SC}))$$

代数优化原则

代数优化是不考虑具体存储路径，仅根据关系代数等价变换优化表达式。

核心原则：

1. 尽可能早地执行选择 σ ，减少行数。
2. 尽可能早地执行投影 π ，减少列数，但不能删掉后续连接、选择、输出需要的属性。
3. 把选择和笛卡尔积合并成连接。

4. 投影和选择尽量与前后的操作合并，减少中间结果。
5. 找出公共子表达式，避免重复计算。

为什么避免笛卡尔积：

若 R 有 n 行， S 有 m 行：

$$|R \times S| = n \cdot m$$

中间结果可能巨大。连接本质上是带条件的笛卡尔积：

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

所以应尽量先选择、投影，再连接。

常用等价变换

选择串接：

$$\sigma_{F_1 \wedge F_2}(E) \equiv \sigma_{F_1}(\sigma_{F_2}(E))$$

选择交换：

$$\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_2}(\sigma_{F_1}(E))$$

选择下推到连接：

$$\sigma_{F_R}(R \bowtie S) \equiv (\sigma_{F_R}(R)) \bowtie S$$

其中 F_R 只涉及 R 的属性。

选择和笛卡尔积合并：

$$\sigma_{R.A=S.B}(R \times S) \equiv R \bowtie_{R.A=S.B} S$$

关系表达式优化算法

课件给出的算法思路：

1. 把复杂选择拆成多个选择：

$$\sigma_{F_1 \wedge F_2 \wedge \dots \wedge F_n}(E) \Rightarrow \sigma_{F_1}(\sigma_{F_2}(\dots \sigma_{F_n}(E) \dots))$$

2. 把每个选择尽可能移向语法树叶端。
3. 把每个投影尽可能移向语法树叶端，但保留后续需要属性。
4. 把串接的选择、投影合并，争取一次扫描完成。
5. 把选择和笛卡尔积合并为连接。

物理优化

物理优化是根据存取路径、索引、统计信息和操作算法选择执行策略。

常见选择操作实现：

- 全表扫描：没有索引或选择条件不适合索引时。
- 索引扫描：选择条件涉及有索引属性，尤其是等值条件或范围条件时。

常见连接算法：

算法	适用情况
嵌套循环连接	一个表很小，或外表筛选后很小时
索引嵌套循环连接	内表连接属性有索引
排序归并连接	两表已经按连接属性排序，或排序代价可接受
散列连接	等值连接，且能用哈希把数据较好分桶

物理优化使用时机：

- 选择条件有索引：优先考虑索引。
 - 两表连接属性已排序：优先考虑排序归并连接。
 - 一个关系很小，另一个关系连接属性有索引：优先考虑索引嵌套循环。
 - 等值连接且无可用排序/索引优势：可考虑散列连接。
-

第八章 事务管理

考试注意事项：并发控制、故障恢复、故障日志、REDO/UNDO、锁机制。

事务与 ACID

事务是数据库中一组操作构成的逻辑工作单位，要么全部成功，要么全部失败。

事务通常以：

```
BEGIN TRANSACTION;
COMMIT;
ROLLBACK;
```

相关语句表示。

ACID：

特性	含义
原子性 Atomicity	事务中操作要么全做，要么全不做
一致性 Consistency	事务执行前后数据库都应处于一致状态
隔离性 Isolation	并发事务之间不能互相干扰
持久性 Durability	事务提交后结果应永久保存

恢复技术保证故障时的原子性和持久性；并发控制保证并发执行时的隔离性和一致性。

故障分类

故障类型	含义	恢复思路
事务故障	单个事务未到正常终点就失败	撤销该事务，做 UNDO
系统故障	系统停止运行，内存内容丢失，磁盘未坏	对未完成事务 UNDO，对已提交但可能未写盘事务 REDO
介质故障	磁盘等外存损坏	装入备份，再利用日志 REDO 已提交事务

日志与先写日志原则

日志用于记录事务对数据库的更新操作，是恢复的基础。

常见日志内容：

- 事务开始标记 BEGIN TRANSACTION 。
- 事务结束标记 COMMIT 或 ROLLBACK 。
- 更新操作记录：事务标识、数据项、旧值、新值。

先写日志原则：

先写日志，再写数据库

原因：如果先改数据库但日志没写，故障后系统不知道如何撤销或重做该更新。

恢复策略

事务故障恢复

只涉及一个失败事务。

步骤：

1. 反向扫描日志，找到该事务的更新操作。
2. 对每个更新执行逆操作，即把“更新前的值”写回数据库。
3. 直到读到该事务的开始标记。

本质：

UNDO

系统故障恢复

重启后自动完成：

1. 正向扫描日志。
2. 有 `BEGIN` 且有 `COMMIT` 的事务放入 REDO 队列。
3. 有 `BEGIN` 但没有 `COMMIT` 的事务放入 UNDO 队列。
4. 对 UNDO 队列事务反向撤销。
5. 对 REDO 队列事务正向重做。

记忆：

未提交 $\Rightarrow$ UNDO

已提交 $\Rightarrow$ REDO

介质故障恢复

步骤：

1. 装入最近的数据库备份。
2. 装入相应日志。
3. 重做备份后已提交的事务。

介质故障通常需要 DBA 介入。

检查点

检查点用于减少恢复时扫描日志的范围，提高恢复效率。

检查点记录通常包括：

- 检查点时正在执行的事务清单。
- 这些事务最近日志记录的地址。

恢复时：

1. 找到最后一个检查点。
2. 从检查点开始扫描日志。
3. 建立 UNDO-LIST 和 REDO-LIST。
4. 对 UNDO-LIST 执行 UNDO，对 REDO-LIST 执行 REDO。

典型判断：

- 检查点前已提交，且数据已写盘：通常不需要 REDO。
- 检查点后提交：需要 REDO。
- 故障时未提交：需要 UNDO。

并发执行可能导致的问题

问题	含义
丢失修改	两个事务写同一数据，后写覆盖先写
读脏数据	一个事务读到另一个未提交事务写的的数据
不可重复读	同一事务两次读同一数据，中间被其他事务修改
幻读	同一事务按条件读一批记录，中间被其他事务插入/删除符合条件的记录

并发控制目标：让并发调度的结果等价于某个串行调度。

可串行化与前趋图

调度正确性的标准是可串行化。

冲突操作满足：

- 属于不同事务。
- 访问同一数据项。
- 至少有一个是写操作。

冲突包括：

$$R_i(X) \text{ 与 } W_j(X)$$

$$W_i(X) \text{ 与 } R_j(X)$$

$$W_i(X) \text{ 与 } W_j(X)$$

前趋图判定方法：

1. 每个事务作为一个结点。
2. 若 T_i 的某个操作先于 T_j 的冲突操作，则画边：

$$T_i \rightarrow T_j$$

3. 若图中有环，则不可冲突串行化。
4. 若图中无环，则可冲突串行化，拓扑序就是等价串行顺序。

锁类型

锁	又称	作用
X 锁	排它锁、写锁	加锁事务可读可写，其他事务不能再加任何锁
S 锁	共享锁、读锁	加锁事务可读不可写，其他事务可再加 S 锁但不能加 X 锁
U 锁	更新锁	先声明可能更新，之后需要写时升级为 X 锁，提高并发度

相容性核心：

- SS 与 SS 相容。
- SS 与 SX 不相容。
- X 与任何锁都不相容。

三级加锁协议

协议	规则	能解决的问题
一级加锁协议	修改前加 X 锁，事务结束才释放	防止丢失修改
二级加锁协议	一级 + 读前加 S 锁，读完即可释放	防止丢失修改、读脏数据
三级加锁协议	二级 + 读锁保持到事务结束	防止丢失修改、读脏数据、不可重复读

两段锁协议 2PL

两段锁协议把事务加锁过程分为两个阶段：

1. 扩展阶段：只能申请锁，不能释放锁。
2. 收缩阶段：只能释放锁，不能再申请锁。

结论：

若所有事务都遵守两段锁协议，则并发调度一定是可串行化的。

注意：

- 2PL 保证可串行化。
- 2PL 不保证不会死锁。
- 一次封锁法一定满足 2PL， 但 2PL 不要求一次性拿到所有锁。

活锁与死锁

活锁：某事务长期等待，总是被其他事务插队。解决思路：先来先服务。

死锁：事务之间形成循环等待。

死锁处理：

- 预防：一次封锁法、顺序封锁法。
 - 检测：构造事务等待图，若有环则死锁。
 - 解除：选择代价较小的事务撤销，释放其锁。
-

第九章 数据库完整性和安全性

考试注意事项：基本概念掌握。

完整性与安全性的区别

完整性：保证数据正确、有效、相容。

安全性：防止非法用户非法使用数据库，避免数据泄露、篡改、破坏。

对比：

概念	关注点	例子
完整性	数据是否正确	成绩必须在 0-100；外键必须引用已有学生
安全性	谁能访问/修改数据	普通学生不能修改成绩表

完整性约束分类

完整性约束按作用对象可分为：

- 列约束：类型、长度、取值范围、是否允许空值。
- 元组约束：同一元组各列之间的关系。
- 关系约束：元组之间、关系之间的约束。

按状态变化可分为：

- 静态约束：数据库某一确定状态必须满足的约束。
- 动态约束：数据库从一种状态变到另一种状态时，新旧值之间必须满足的约束。

六类完整性约束：

类型	例子
静态列级约束	年龄不能为负，成绩在 0-100
静态元组约束	开始日期不能晚于结束日期
静态关系约束	实体完整性、参照完整性、函数依赖
动态列级约束	工资修改后不能低于原工资
动态元组约束	新工资不得低于原工资加某个计算值
动态关系约束	事务一致性、原子性

完整性控制机制

DBMS 的完整性控制应包括：

1. 定义功能：允许用户定义完整性约束。
2. 检查功能：检查操作是否违反约束。
3. 违约处理：拒绝操作、级联处理、置空、回滚等。

检查时机：

- 立即执行约束：语句执行后立即检查。
- 延迟执行约束：事务提交前统一检查。

参照完整性处理策略

删除被引用关系元组时，常见策略：

- **CASCADE**：级联删除引用关系中的相关元组。
- **RESTRICT**：若存在引用元组，则拒绝删除。
- **SET NULL**：删除被引用元组，同时把引用关系中对应外键置空。

例子：

Student(Sno, Sname)

SC(Sno, Cno, Grade)

若删除某个学生：

- **CASCADE**：同时删除该学生所有选课记录。
- **RESTRICT**：若该学生有选课记录，就不允许删除学生。
- **SET NULL**：把选课记录中的学号置空，但前提是外键允许空值。

完整性约束说明方式

隐含约束：

- 数据类型、长度。
- 主键。
- 外键。
- **NOT NULL**。

显式约束：

- **CHECK** 子句。
- 断言 **ASSERTION**。
- 触发器 **TRIGGER**。
- 存储过程。

例子：

```
CHECK (Grade IS NULL OR Grade BETWEEN 0 AND 100)
```

表示成绩要么为空，要么在 0 到 100 之间。

安全性控制

常见安全措施：

1. 用户标识与鉴别：用户名、口令等。
2. 存取控制：定义用户权限并检查请求是否合法。
3. 视图机制：通过视图隐藏敏感数据。
4. 审计：记录用户操作，事后追踪非法访问。
5. 加密：保护敏感数据。

自主存取控制与强制存取控制

自主存取控制 DAC：

- 用户权限由授权机制控制。
- SQL 中常用 `GRANT`、`REVOKE`。
- 用户可在允许时把权限转授给别人。

强制存取控制 MAC：

- 系统为主体和客体分配安全级别/密级。
- 用户不能自主改变访问规则。
- 安全级别更高，适合严格保密场景。

SQL 授权例子：

```
GRANT SELECT, UPDATE(Grade) ON TABLE SC TO Wang;
```

```
REVOKE UPDATE(Grade) ON TABLE SC FROM Wang;
```

考前优先级建议

1. 第一章：背清楚 DB、DBMS、DBS、三级模式、两级映像、逻辑/物理独立性、数据模型三要素。
2. 第二章：重点练关系代数。尤其是选择、投影、连接、差、除、自连接和“全部/所有”查询。
3. 第三章：熟练写 `SELECT`、`INSERT`、`DELETE`、`UPDATE`，注意空值、分组、连接、子查询。
4. 第四章：反复练属性闭包、候选键、最小覆盖、无损连接判定、3NF/BCNF 分解。
5. 第五章：按“先 ER 图、再关系模式”答题； $1:N$ 合并到 N 端， $M:N$ 额外建联系表。
6. 第七章：记住语法树对应关系、选择/投影下推、笛卡尔积合并为连接，以及常见物理连接算法使用时机。
7. 第八章：掌握事务 ACID、故障分类、日志 REDO/UNDO、检查点、并发异常、锁和 2PL。
8. 第九章：区分完整性与安全性，掌握参照完整性策略、授权、视图、审计等基本概念。